

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica

2 de mayo de 2026

Ignacio Malmierca
113695

Nadeska Millan
110779

Samuel Herrera
112861

Introduction

En el presente trabajo práctico se aborda el diseño y análisis de un algoritmo de Programación Dinámica, una técnica fundamental para la resolución de problemas de optimización con subestructura óptima y subproblemas superpuestos. El objetivo principal es comprender en qué contextos este enfoque permite obtener soluciones óptimas y cómo justificar formalmente su corrección mediante una ecuación de recurrencia.

Para ello, se propone el siguiente problema: se debe asistir a Scaloni en la planificación del entrenamiento de la selección durante los próximos n días previos al mundial. Cada día de entrenamiento posee una ganancia predefinida e_i , sin embargo, la energía disponible de los jugadores decrece a medida que acumulan días consecutivos de trabajo: el j -ésimo día entrenado sin descanso, los jugadores disponen de una energía s_j , con $s_1 \geq s_2 \geq \dots \geq s_n$. La ganancia efectiva de un día entrenado resulta entonces del mínimo entre la energía disponible y la ganancia predefinida de ese día, Scaloni puede optar por hacer descansar a los jugadores un día, renovando su energía por completo, aunque a costa de resignar la ganancia de ese día. El desafío consiste en determinar qué días conviene entrenar y qué días conviene descansar, de modo tal que la ganancia total acumulada a lo largo de los n días sea máxima.

A lo largo del trabajo se realizará un análisis formal del problema, se planteará la ecuación de recurrencia correspondiente y se propondrá un algoritmo de Programación Dinámica para su resolución. Asimismo, se demostrará la optimalidad de dicha ecuación y se estudiará la complejidad tanto del algoritmo de programación dinámica como del algoritmo de reconstrucción de la solución.

1. Análisis del problema y formulación por Programación Dinámica

El problema consiste en determinar, para una secuencia de n días, cuáles días conviene entrenar y cuáles descansar, con el objetivo de maximizar la ganancia total obtenida.

Para cada día i se conoce una ganancia potencial E_i , asociada al entrenamiento de dicho día. Sin embargo, la ganancia real depende también de la energía disponible de los jugadores, la cual está determinada por la cantidad de días consecutivos que han entrenado desde el último descanso.

Sea $S = (S_1, S_2, \dots, S_n)$ la secuencia de energía disponible, donde S_j representa la energía disponible luego de haber entrenado j días consecutivos desde el último descanso, cumpliéndose que:

$$S_1 \geq S_2 \geq \dots \geq S_n$$

Si se decide entrenar el día i , la ganancia obtenida está dada por:

$$\min(S_j, E_i)$$

donde j es la cantidad de días consecutivos que se ha entrenado hasta ese momento.

Alternativamente, se puede optar por descansar, lo cual implica no obtener ganancia en ese día, pero reiniciar la energía, de modo que el próximo día se contará nuevamente con energía S_1 .

Para modelar el problema mediante programación dinámica, se define la siguiente función:

$EO[i][j]$ = máxima ganancia que se puede obtener desde el día i hasta el día n , habiendo entrenado j días consecutivos.

La ecuación de recurrencia se define considerando las dos decisiones posibles en cada estado:

$$EO[i][j] = \max(\min(S_j, E_i) + EO[i+1][j+1], EO[i+1][1])$$

donde:

- El primer término corresponde a la decisión de entrenar el día i , obteniendo la ganancia del día i y continuando con $j+1$ días consecutivos de entrenamiento.
- El segundo término corresponde a la decisión de descansar, lo que reinicia la cuenta de días consecutivos de entrenamiento.

La condición base de la recurrencia es:

$$EO[n+1][j] = 0 \quad \forall j$$

ya que, una vez superado el último día, no se puede obtener ganancia adicional.

Cabe destacar que, en la implementación en Python, los arreglos se indexan desde 0. Por lo tanto, la expresión $\min(S_j, E_i)$ se implementa como $\min(S[j-1], E[i-1])$, manteniendo la equivalencia con la formulación matemática.

El algoritmo de programación dinámica propuesto construye una tabla bidimensional que almacena los valores de $EO[i][j]$ para todos los posibles estados, permitiendo obtener finalmente la ganancia máxima buscada.

2. Demostración de la optimalidad de la ecuación de recurrencia

Se demostrará que la ecuación de recurrencia planteada permite obtener la solución óptima al problema mediante el principio de optimalidad de la programación dinámica.

La recurrencia definida depende del día actual de entrenamiento i y de la cantidad de días consecutivos entrenados desde el último descanso j . Al ser implementada mediante un enfoque *bottom-up*, se almacenan los resultados de los subproblemas en una tabla, evitando recalculados estados previamente resueltos.

En cada estado se consideran todas las decisiones posibles (entrenar o descansar) y se selecciona aquella que maximiza la ganancia total. Este enfoque es válido debido al principio de optimalidad, el cual establece que toda solución óptima está compuesta por soluciones óptimas de sus subproblemas.

Demostración por contradicción

Sea la ecuación de recurrencia:

$$EO[i][j] = \max(\min(S_j, E_i) + EO[i+1][j+1], EO[i+1][1])$$

Supóngase por contradicción que existe una solución óptima $A2[i][j]$ tal que:

$$A2[i][j] > EO[i][j]$$

Denotemos a $EO[i][j]$ como $A1[i][j]$, es decir, la solución obtenida por la recurrencia.

Se analizarán los posibles casos de decisión en el estado (i, j) .

Caso 1: Entrenar

Si la solución óptima decide entrenar el día i , entonces se cumple que:

$$A2[i][j] = \min(S_j, E_i) + A2[i+1][j+1]$$

Por hipótesis inductiva, se asume que los subproblemas están correctamente resueltos:

$$A2[i+1][j+1] = EO[i+1][j+1]$$

Por lo tanto:

$$A2[i][j] = \min(S_j, E_i) + EO[i+1][j+1]$$

Sin embargo, por definición de la recurrencia:

$$EO[i][j] \geq \min(S_j, E_i) + EO[i+1][j+1]$$

Entonces:

$$EO[i][j] \geq A2[i][j]$$

lo cual contradice la suposición inicial de que $A2[i][j] > EO[i][j]$.

Caso 2: Descansar

Si la solución óptima decide descansar en el día i , entonces:

$$A2[i][j] = A2[i+1][1]$$

Por hipótesis inductiva:

$$A2[i+1][1] = EO[i+1][1]$$

Por lo tanto:

$$A2[i][j] = EO[i+1][1]$$

Y por definición de la recurrencia:

$$EO[i][j] \geq EO[i+1][1]$$

lo que implica:

$$EO[i][j] \geq A2[i][j]$$

lo cual contradice nuevamente la suposición inicial.

En ambos casos se obtiene una contradicción, por lo tanto la suposición de que existe una solución mejor que la obtenida por la recurrencia es falsa.

En consecuencia, se concluye que la ecuación de recurrencia calcula correctamente la solución óptima del problema:

$$EO[i][j] = OPT[i][j]$$

3. Algoritmo y análisis de complejidad

3.1. Algoritmo de programación dinámica

El algoritmo propuesto construye una tabla bidimensional EO de dimensiones $(n+2) \times (n+2)$, donde cada entrada $EO[i][j]$ representa la máxima ganancia posible desde el día i hasta el día n , habiendo entrenado j días consecutivos desde el último descanso.

La tabla se completa en forma *bottom-up*, comenzando desde el día n hacia el día 1, de manera que al momento de calcular cada estado $EO[i][j]$, los valores necesarios para los subproblemas ya han sido previamente computados.

En cada estado se consideran dos posibles decisiones:

- **Entrenar:** se obtiene una ganancia igual a $\min(S_j, E_i)$ y se continúa al estado $(i+1, j+1)$.
- **Descansar:** no se obtiene ganancia en el día actual y se pasa al estado $(i+1, 1)$.

El valor óptimo en cada celda se obtiene tomando el máximo entre ambas opciones.

3.1.1 Pseudocódigo del algoritmo

```
1 def entrenamiento(E,S):
2     cantidad_dias = len(E)
3     entrenamiento_optimo = []
4     for i in range (cantidad_dias + 2):
5         fila = []
6         for j in range (cantidad_dias + 2):
7             fila.append(0)
8         entrenamiento_optimo.append(fila)
9     for dia_actual in range (cantidad_dias,0,-1):
10        for dias_seguidos in range(1, cantidad_dias + 1):
```

```
11     ganancia_entrenar = min(S[dias_seguidos - 1], E[dia_actual-1]) +  
    entrenamiento_optimo[dia_actual + 1][dias_seguidos + 1]  
12     ganancia_descansar = entrenamiento_optimo[dia_actual + 1][1]  
13     if ganancia_entrenar > ganancia_descansar:  
14         entrenamiento_optimo[dia_actual][dias_seguidos] = ganancia_entrenar  
15     else:  
16         entrenamiento_optimo[dia_actual][dias_seguidos] =  
    ganancia_descansar  
17     return entrenamiento_optimo
```

El resultado final de la ganancia máxima se obtiene en la posición $EO[1][1]$, que representa comenzar desde el primer día sin haber acumulado días de entrenamiento previos.

3.2. Algoritmo de reconstrucción

Una vez construida la tabla EO , es posible reconstruir una solución óptima recorriendo nuevamente los días desde 1 hasta n , tomando en cada paso la decisión que haya dado lugar al valor almacenado en la tabla.

Se mantiene un contador de días consecutivos entrenados y, para cada día, se comparan las dos opciones posibles (entrenar o descansar), eligiendo aquella que coincide con el valor almacenado en la tabla.

3.2.1 Pseudocódigo de reconstrucción

```
1 def reconstruir_entrenamiento(entrenamiento_optimo, E, S):  
2     cantidad_dias = len(E)  
3     dias_seguidos = 1  
4     decisiones = []  
5     for dia_actual in range(1, cantidad_dias + 1):  
6         ganancia_entrenar = min(S[dias_seguidos - 1], E[dia_actual - 1]) +  
    entrenamiento_optimo[dia_actual + 1][dias_seguidos + 1]  
7         ganancia_descansar = entrenamiento_optimo[dia_actual + 1][1]  
8         if ganancia_entrenar > ganancia_descansar:  
9             decisiones.append("Entrenar")  
10            if dias_seguidos < len(S):  
11                dias_seguidos += 1  
12        else:  
13            decisiones.append("Descansar")  
14            dias_seguidos = 1  
15    return decisiones
```

3.3. Análisis de complejidad

3.3.1. Complejidad del algoritmo de programación dinámica

La tabla EO tiene dimensiones $(n + 2) \times (n + 2)$, lo que implica un total de $O(n^2)$ estados.

Cada estado se calcula en tiempo constante, ya que únicamente se realizan operaciones aritméticas simples y accesos a posiciones previamente computadas de la tabla.

Por lo tanto, la complejidad temporal del algoritmo de programación dinámica es:

$$O(n^2)$$

En cuanto al espacio utilizado, la matriz EO también requiere $O(n^2)$ memoria.

3.3.2. Complejidad del algoritmo de reconstrucción

El algoritmo de reconstrucción recorre los n días exactamente una vez. En cada iteración realiza un número constante de operaciones.

Por lo tanto, la complejidad temporal del algoritmo de reconstrucción es:

$$O(n)$$

El espacio adicional utilizado es también $O(n)$, correspondiente a la lista de decisiones generada.

4. Ejemplos

Caso	n	Descripción	Plan óptimo	Ganancia
Siempre entrenar	3	Energía decrece levemente ($10 \rightarrow 9 \rightarrow 8$). Nunca conviene sacrificar un día.	E, E, E	27
Descanso frecuente	5	Energía cae abruptamente al 2do día consecutivo ($10 \rightarrow 1$). Conviene alternar.	E, D, E, D, E	30
Energía siempre mayor	5	$S_j \geq E_i$ siempre. La ganancia real es siempre E_i , conviene entrenar todos los días.	E, E, E, E, E	37
Energía cae rápido	5	Energía cae a 2 desde el 2do día ($9 \rightarrow 2$). Entrenar seguido no compensa.	E, D, E, D, E	24
Caso mixto	10	Ganancias y energías variadas. El algoritmo intercala descansos de forma no trivial.	E, E, D, E, E, E, D, E, E, E	47
Ganancias crecientes	5	Días más rentables al final, pero la energía ya decreció. Conviene descansar antes del último día.	E, E, E, D, E	11
Energía muy baja	3	Energía constante en 1. La ganancia efectiva es siempre $\min(1, E_i) = 1$. Se entrena todos los días.	E, E, E	3

Cuadro 1: Casos de prueba propios con sus resultados óptimos. E = Entrenar, D = Descansar.

5. Conclusiones